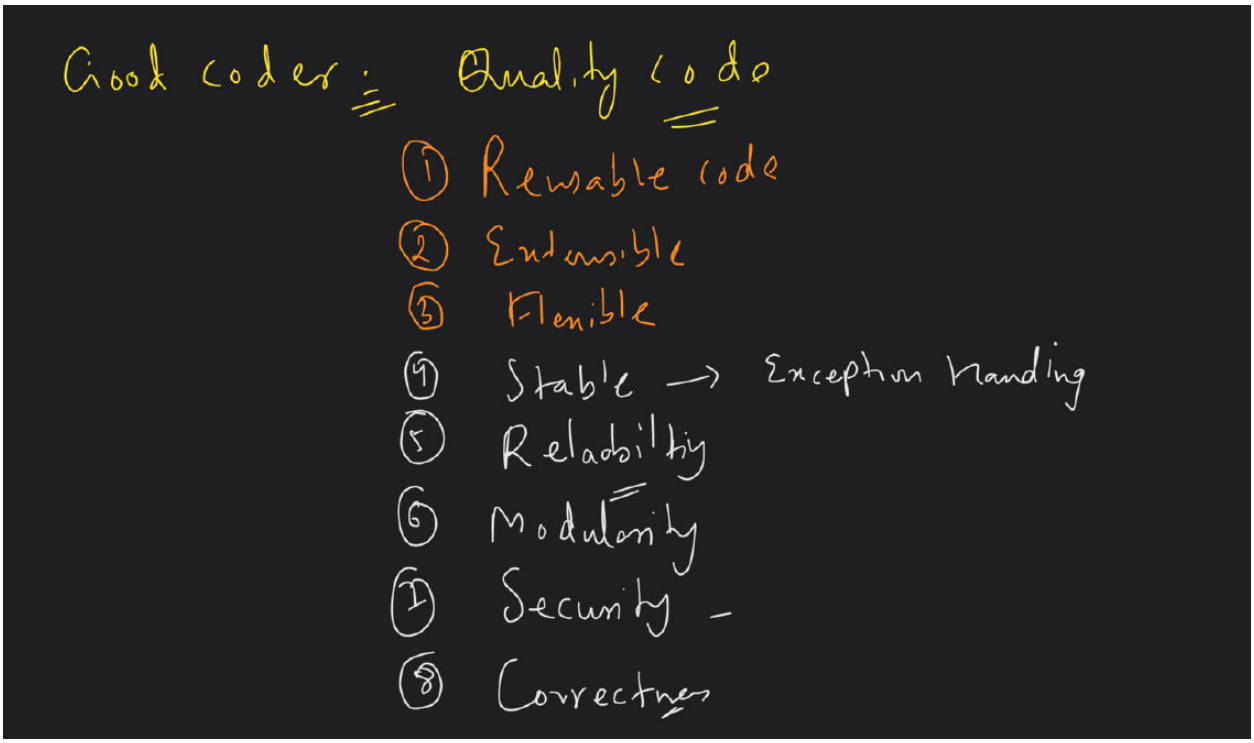
Solid Principles

> - Lalit Sharma

Qualities of Good Code



> The programmer who adheres to these protocols is known as a good coder.

Reusable Code

Reusable code refers to the practice of writing code in a way that allows it to be reused in various parts of the software or in multiple projects. This is achieved by creating modules, classes, or functions that perform specific tasks and can be easily integrated into different parts of the software.

Extensible

Extensible code is designed with future growth in mind. It's structured in a way that new functionality can be added with little effort.

Flexible

Flexible code is code that can easily be changed. This involves writing code that is easy to read and understand, keeping functions and classes small, and keeping code DRY (Don't Repeat Yourself). This makes it easier to make changes in the future.

Stable

Stable code is code that does not easily break or cause other parts of the software to break when changes are made. This involves rigorous testing and adopting practices such as defensive programming, which anticipates and handles potential issues before they become problems. e.g: Exception Handling make code stable.

Reliability

Reliable code is code that consistently performs as expected and produces the desired results. It is free of bugs and errors, and it handles unexpected inputs or situations gracefully.

Modularity

Modular code refers to the process of subdividing a computer program into separate sub-programs, known as modules. Each module performs a specific task and operates independently of the other modules.

Security

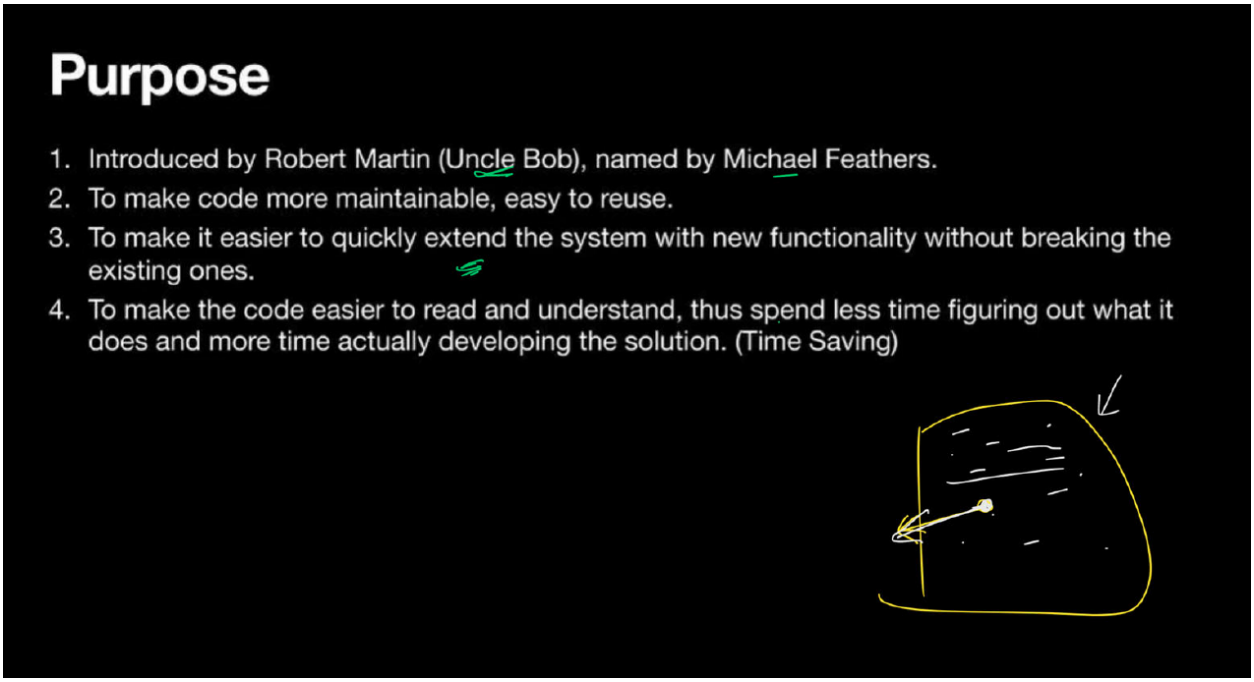
Secure code is code that is written with security in mind. It means that the code is free from vulnerabilities that could be exploited by attackers, and includes practices like input validation, using secure protocols, and adhering to the

principle of least privilege. Simple example would be Encapsulation(Access Modifiers maintain security of class)

Correctness

Correct code is code that accomplishes its intended purpose. This means that the code not only produces the desired output, but does so in the expected manner. Correctness can be ensured through a combination of careful design, thorough testing, and rigorous code reviews.

> Your code can embody all these qualities by adhering to five principles : S O L I D

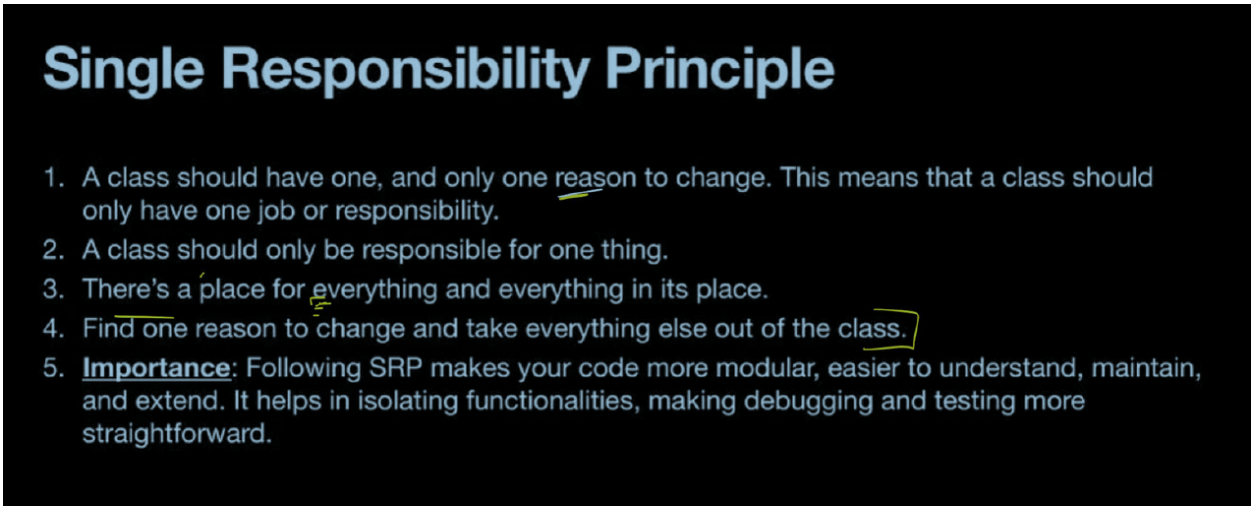


Example for #3 {Purpose}

Consider an application with a payment module that supports payment via Debit Card and Credit Card. When a new payment method, UPI, was added by a developer, the existing payment methods (Debit and Credit Card) started to malfunction. This is because the code for the new payment method was not properly isolated and was interfering with the existing code. This situation is a violation of the principle of extensibility, as the system should allow for the addition of new functionality without breaking existing functionality.

Single Responsibility Principle

The Single Responsibility Principle (SRP) is a computer programming principle that states that every module, class, or function should have responsibility over a single part of the functionality provided by the software, and that responsibility should be entirely encapsulated by the class. This principle gives your program better organisation and facilitates understanding and maintenance.



Use Case 1

SRPVoilation.java

```java
package solidPrinciples.srp;

public class SRPVoilation {

    /**
     * Created a UserInfo class that
     * should have user information
     * but also have printErr method to print errors
     * Hence, it is voilating the Single Responsibility Principle
     **/

    class UserInfo{

        void addUser(String name){
            System.out.println("User name is : " + name);
        }

        void printErr(String error){
            System.out.println("Printing error : " +  error);
        }
    }

}
```

Correcting the Single Responsibility Principle (SRP) violation in the aforementioned screenshot.

```
    class UserInfo{

        void addUser(String name){
            System.out.println("User name is : " + name);
        }

//        void printErr(String error){
//            System.out.println("Printing error : " +  error);
//        }
    }

    /**
     * To correct it, let's create a new class that should only be responsible for logging
     * and remove printErr method from UserInfo class
     * **/

    class Logger{

        void printErr(String error){
            System.out.println("Printing error : " +  error);
        }

    }

}
```

Use Case 2

```
packagesolidPrinciples.srp;

importjava.util.ArrayList;
importjava.util.List;

/**
 * Order class should be responsible for
 * functionality related to Orders only
 * But here we can see it is also accounting for
 * Payments. Hence, voilate SRP here
 * **/


classOrder {
```

```
privateList<String>items;
privateList<Integer>quantities;
privateList<Double>prices;
privateStringstatus;

publicOrder() {
this.items=newArrayList<>();
this.quantities=newArrayList<>();
this.prices=newArrayList<>();
this.status="open";
    }

public voidaddItem(String name,intquantity,doubleprice) {
items.add(name);
quantities.add(quantity);
prices.add(price);
    }

private doubletotalPrice() {
doubletotal = 0;
for(inti = 0; i <prices.size(); i++) {
            total +=quantities.get(i) *prices.get(i);
        }
returntotal;
    }

public voidpay(String paymentType, String securityCode) {
if("debit".equals(paymentType)) {
            System.out.println("Processing debit payment typ
e");
            System.out.println("Verifying security code: "+ s
ecurityCode);
status="paid";
        }else if("credit".equals(paymentType)) {
            System.out.println("Processing credit payment typ
e");
```

```
            System.out.println("Verifying security code: "+ s
ecurityCode);
status="paid";
        }else{
throw newRuntimeException("Unknown payment type: "+ paymentTy
pe);
        }
    }

public static voidmain(String[] args) {
        Order order =newOrder();
        order.addItem("Keyboard", 1, 50);
        order.addItem("SSD", 1, 150);
        order.addItem("USB cable", 2, 5);

        System.out.println(order.totalPrice());
        order.pay("debit","0372846");
    }
}
```

In the above screenshot, the Order class should only manage functionalities related to orders. However, it is also handling payments associated with orders. Therefore, this violates the Single Responsibility Principle (SRP).

```
packagesolidPrinciples.srp;

importjava.util.ArrayList;
importjava.util.List;

classOrder {
privateList<String>items;
privateList<Integer>quantities;
```

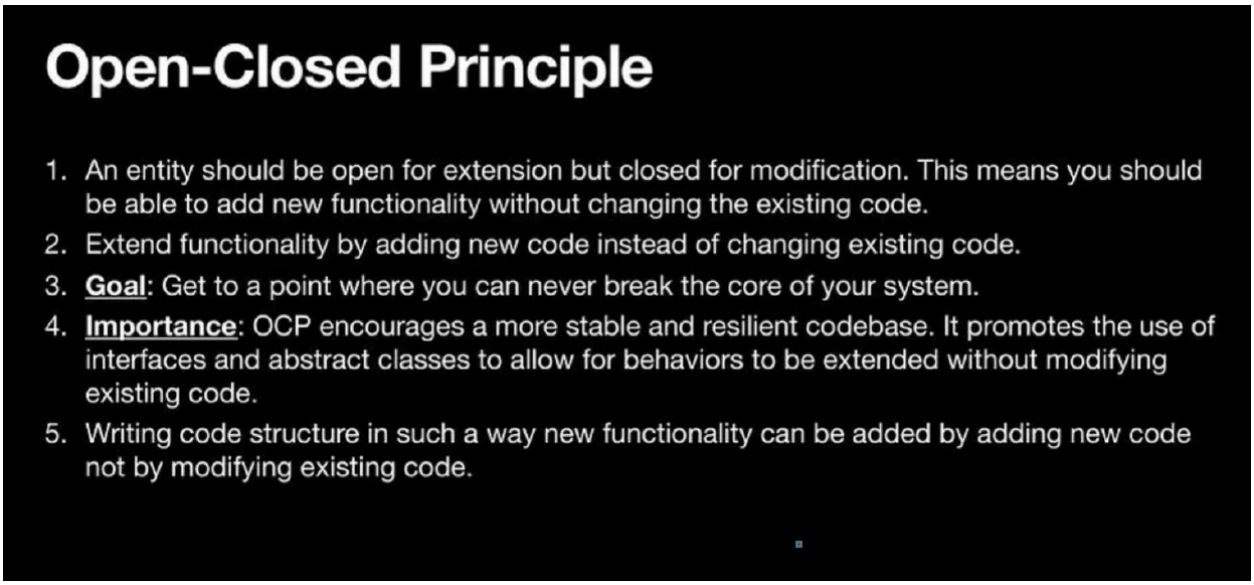
```
privateList<Double>prices;
privateStringstatus;

publicOrder() {
this.items=newArrayList<>();
this.quantities=newArrayList<>();
this.prices=newArrayList<>();
this.status="open";
    }

public voidaddItem(String name,intquantity,doubleprice) {
items.add(name);
quantities.add(quantity);
prices.add(price);
    }

public doubletotalPrice() {
doubletotal = 0;
for(inti = 0; i <prices.size(); i++) {
            total +=quantities.get(i) *prices.get(i);
        }
returntotal;
    }

public voidsetStatus(String status) {
this.status= status;
    }

publicString getStatus() {
returnstatus;
    }
}

/**
* Seggregated the Pyament logic to seperate class
**/
```

```
class PaymentProcessor {
    public void pay(Order order, String securityCode, String
paymentType) {
        if ("debit".equals(paymentType)) {
            System.out.println("Processing debit payment typ
e");
            System.out.println("Verifying security code: " +
securityCode);
            order.setStatus("paid");
        } else if ("credit".equals(paymentType)) {
            System.out.println("Processing credit payment typ
e");
            System.out.println("Verifying nkjfnks security co
de: " + securityCode);
            order.setStatus("paid");
        } else if ("UPI".equals(paymentType)) {
            System.out.println("Processing UPI payment typ
e");
            System.out.println("Verifying security code: " +
securityCode);
            order.setStatus("paid");
        }
    }
}
```

In the above code, the Single Responsibility Principle (SRP) is still violated, as different types of payment methods are contained within the same class. Will discuss on this in detail in the next principle.

Open Closed Principle

The Open Closed Principle (OCP) is the second principle in SOLID and it states that software entities (classes, modules, functions, etc.) should be open for extension, but closed for modification. This means that the design and writing of the code should be done in a way that new functionality can be added with minimum changes to the existing code. The code that is written once should be tested and then it should not be modified; instead, new things should be built on top of it. This makes the system more robust and less error-prone due to modifications in the existing functionality. It significantly improves the maintainability of the system and is one of the key principles of software development.



Use Case 1

```
classPaymentProcessor {
public void pay(Order order, String securityCode, String paym
entType) {
        if("debit".equals(paymentType)) {
          System.out.println("Processing debit payment typ
e");
          System.out.println("Verifying security code: "+ sec
```

```
urityCode);
          order.setStatus("paid");
    }else if("credit".equals(paymentType)) {
          System.out.println("Processing credit payment typ
e");
            System.out.println("Verifying nkjfnks security co
de: "+ securityCode);
          order.setStatus("paid");
      }
    }
}
```

In the above code, Payment has only two types: Debit and Credit. If I want to add a new payment type, I would need to modify the existing pay method, which could potentially impact the existing code.

```
classPaymentProcessor {
public void pay(Order order, String securityCode, String paym
entType) {
        if("debit".equals(paymentType)) {
         System.out.println("Processing debit payment type");
         System.out.println("Verifying security code: "+ secu
rityCode);
         order.setStatus("paid");
    }else if("credit".equals(paymentType)) {
         System.out.println("Processing credit payment typ
e");
         System.out.println("Verifying nkjfnks security code:
"+ securityCode);
         order.setStatus("paid");
    }else if("UPI".equals(paymentType)) {
         System.out.println("Processing UPI payment type");
         System.out.println("Verifying security code: "+ secu
rityCode);
```

```
          order.setStatus("paid");
      }
     }
}
```

After implementing UPI payment, you may notice that it violates both the Single Responsibility Principle (SRP) and the Open/Closed Principle (OCP). Since Debit can have its own functionality, it should be in a different class, which addresses SRP. However, all payment methods are related, so they should be connected. To resolve this, we can introduce the concept of Interface/Abstract classes to adhere to both OCP and SRP.

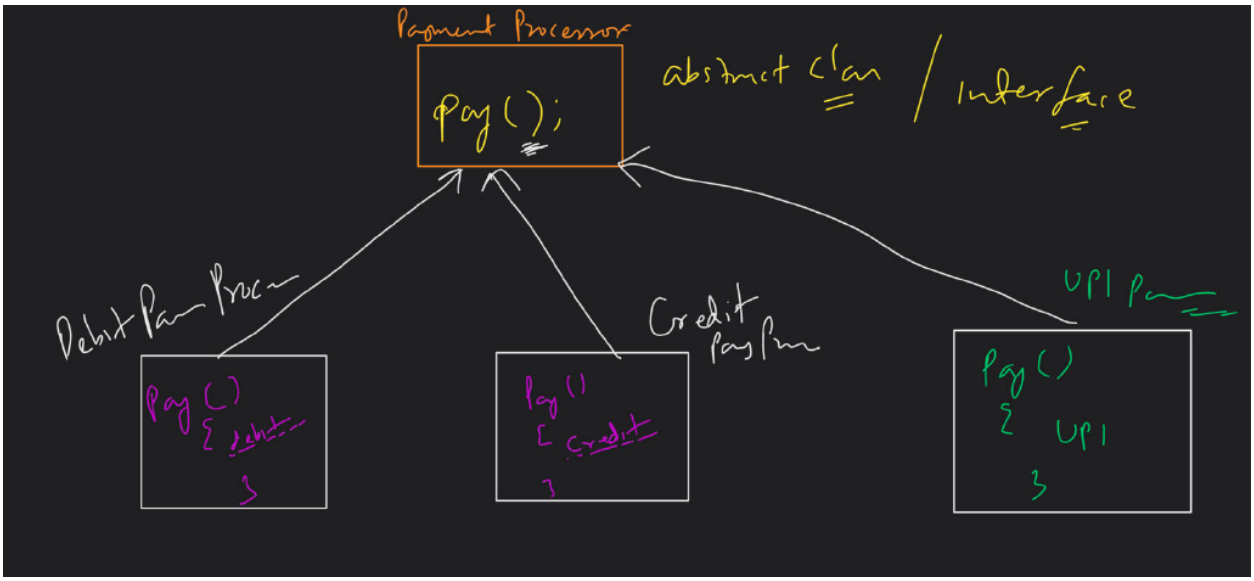
```
interface PaymentProcessor {
    voidpay(Order order, String securityCode);
}
```

```
class DebitPaymentProcessor implements PaymentProcessor {
public voidpay(Order order, String securityCode) {
        System.out.println("Processing debit payment type");
        System.out.println("Verifying security code: "+ secur
ityCode);
        order.setStatus("paid");
    }
}

class CreditPayment Processorimplements PaymentProcessor {
public voidpay(Order order, String securityCode) {
        System.out.println("Processing credit payment type");
        System.out.println("Verifying security code: "+ secur
ityCode);
        order.setStatus("paid");
    }
```

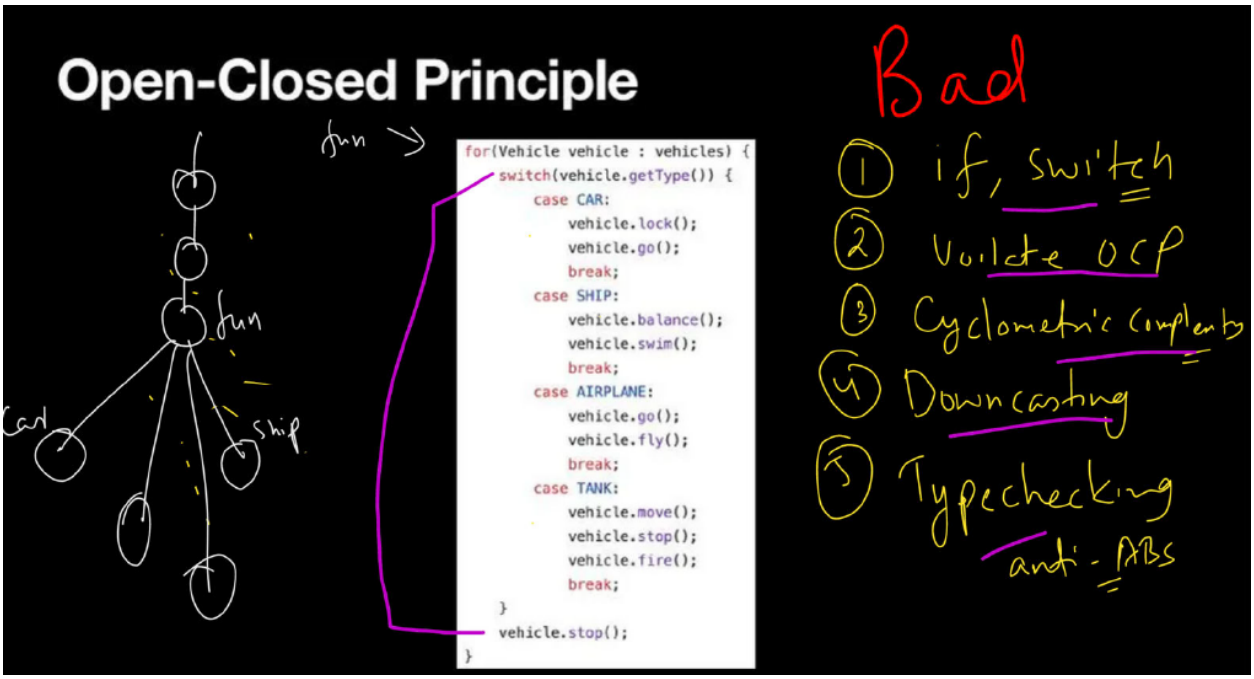
```
}

class UPIPaymentProcessor implements PaymentProcessor {
public voidpay(Order order, String securityCode) {
        System.out.println("Processing UPI payment type");
        System.out.println("Verifying security code: "+ secur
ityCode);
        order.setStatus("paid");
    }
}
```

Finally, We have fixed SRP and OCP over here.

Pictorial Representation of the above code



Qualities of Bad Code



Avoid If-else in the code

Avoiding `if-else` statements in code can be a design principle aimed at improving code readability, maintainability, and flexibility. Instead of using `if-else` statements, you can consider using polymorphism, strategy pattern, or other design patterns to achieve the desired behaviour.

Violate OCP (Open for extension and closed fro Modification)

To violate the Open/Closed Principle (OCP) from a design perspective, you would typically modify existing code rather than extending it. Writing lot of functions in the same class, lot of conditions in the same function leads to violate OCP.

Cyclomatic Complexity

Cyclomatic complexity in code refers to the number of linearly independent paths through the source code. It is a measure of the complexity of the code's control flow. In terms of actual implementation, cyclomatic complexity is typically

calculated based on the number of decision points in the code, such as loops, conditional statements (if, else if, else), and case statements (switch, case).

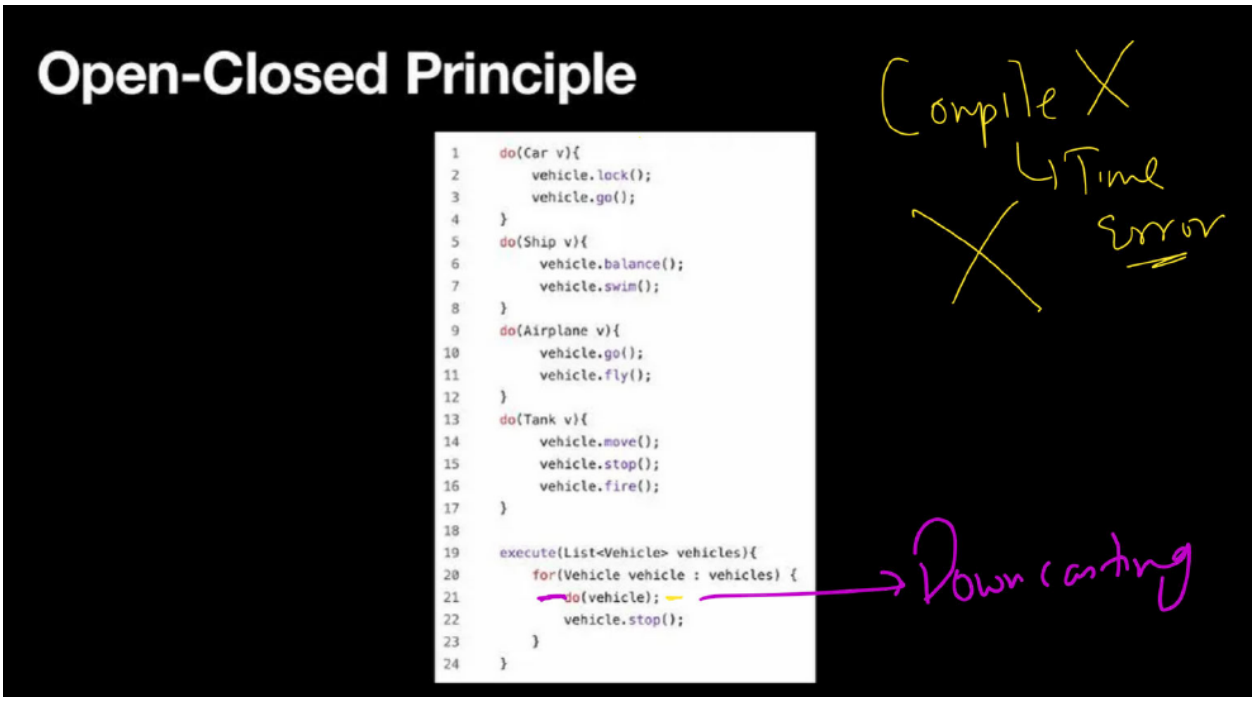
Downcasting

Downcasting in Java refers to the act of casting a reference of a superclass to a subclass. This is typically done when you have a reference to an object of a superclass, but you know that it is actually an object of a subclass. Downcasting allows you to access the methods and fields specific to the subclass.

Avoid Type checking (Instance Of)

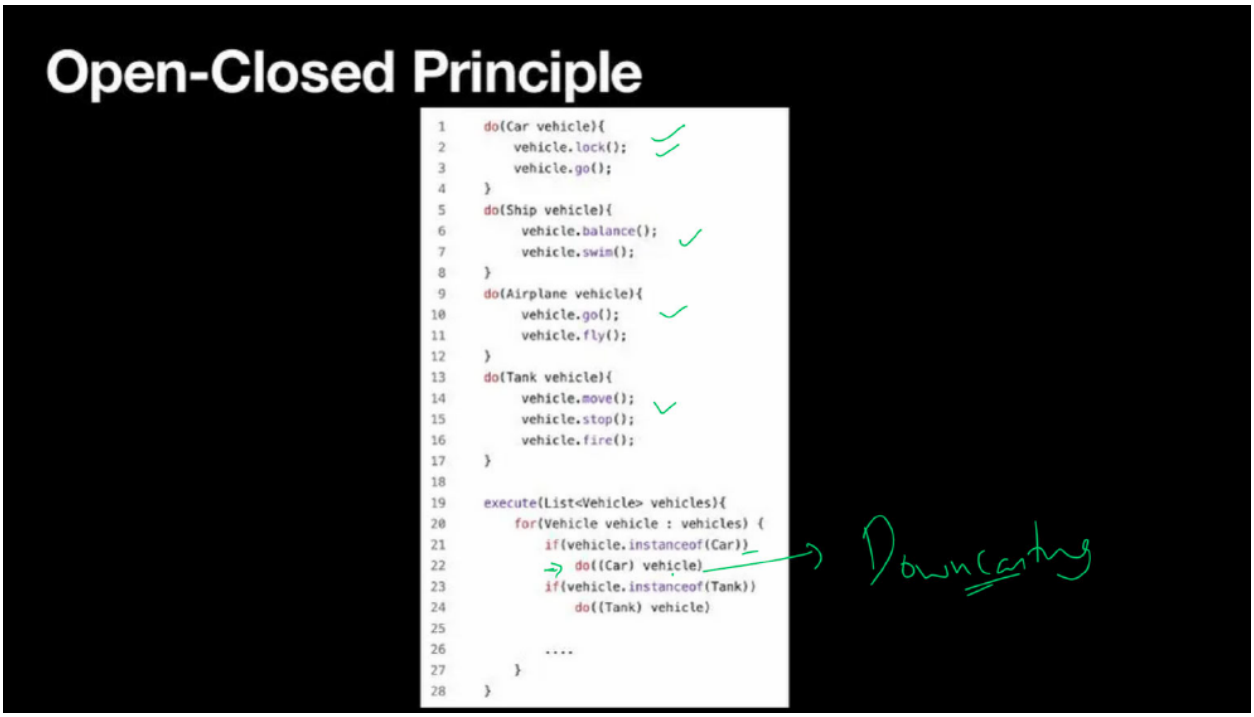
Avoiding the use of `instanceof` in code is often desirable because it can lead to code that is tightly coupled and difficult to maintain. Instead of using `instanceof`, you can often achieve the same functionality using polymorphism and method overriding.

Let’s try to improve the above code

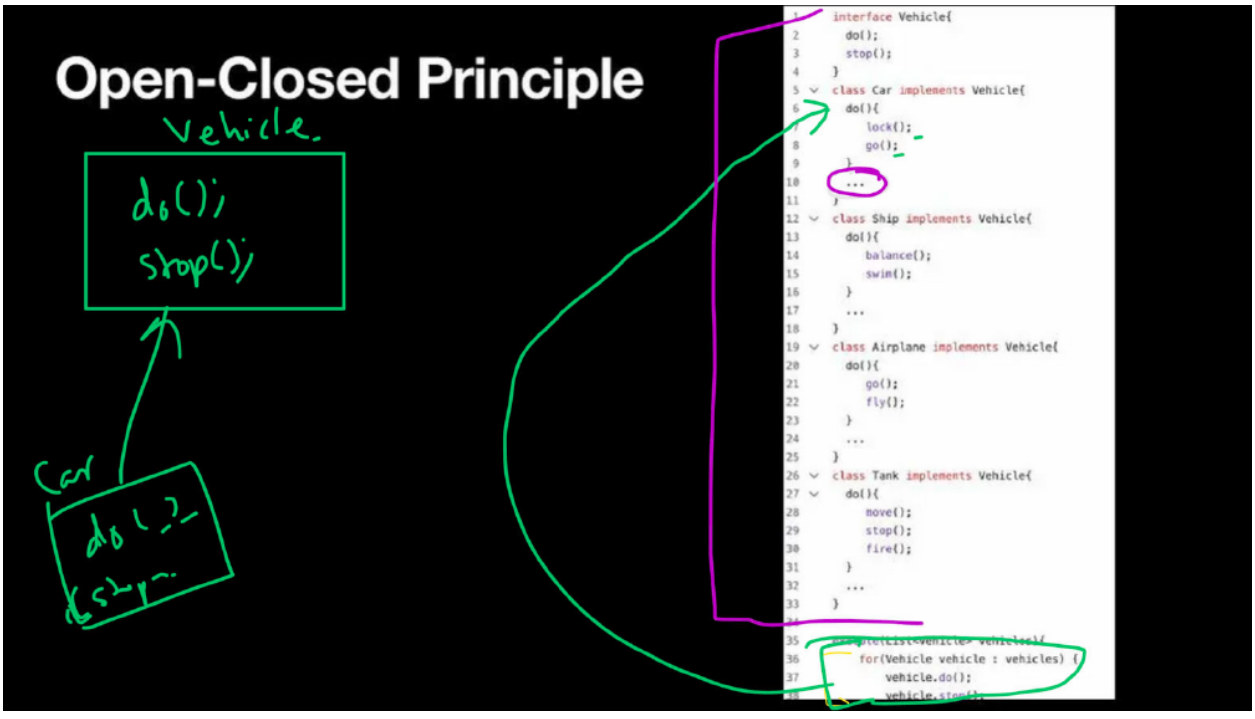


In this code, we've improved the design, but there are two remaining issues (see line no 21):

1. Compile-time error: Since there's no reference to the specific type of the instance, the compiler doesn't know which method to call.
2. Downcasting: To resolve this, we need to check the instance type and then typecast it to call the specific method.



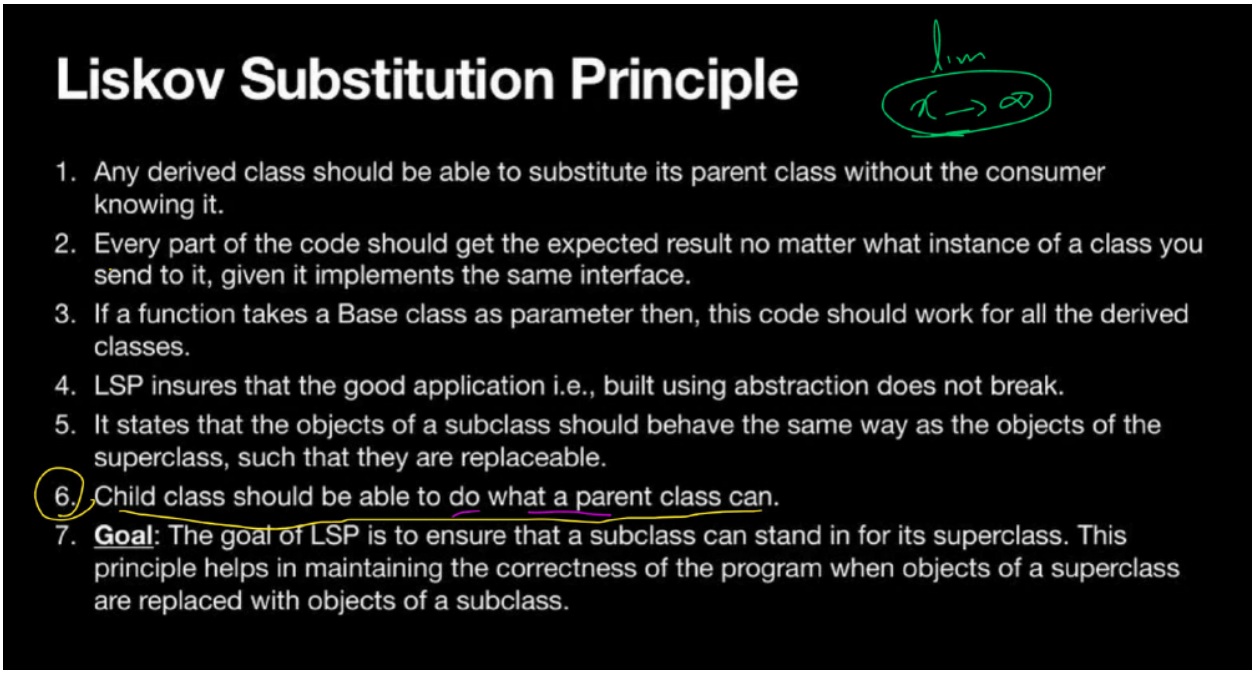
Let’s Improve it more so that it will follow all the principles



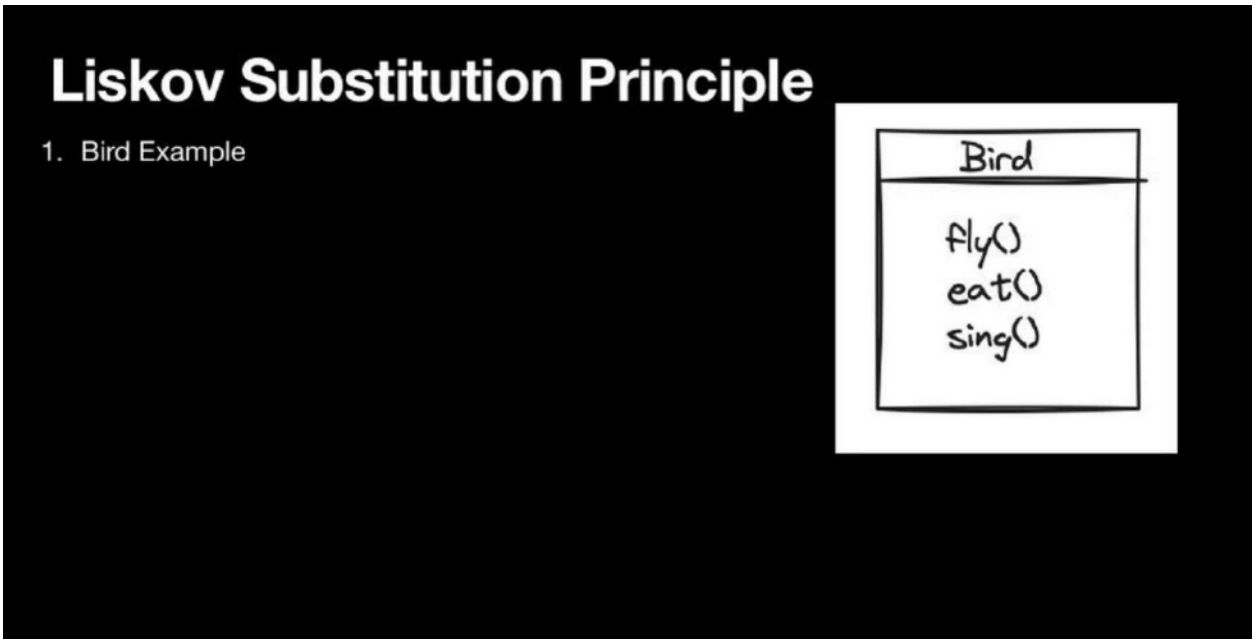
Liskov Substitution Principle

The Liskov Substitution Principle (LSP) is one of the five SOLID principles of object-oriented design, named after Barbara Liskov. It states that objects of a superclass should be replaceable with objects of its subclasses without affecting the correctness of the program. In other words, a subclass should be substitutable for its superclass.

For example, consider a class `Rectangle` with a method `setWidth(int width)` and `setHeight(int height)`, and a subclass `Square` that extends `Rectangle`. If `Square` overrides these methods to set both width and height to the same value (since in a square, both sides are equal), then `Square` violates the Liskov Substitution Principle because it changes the behavior of the superclass `Rectangle`.



Use Case 1 : Bird Example

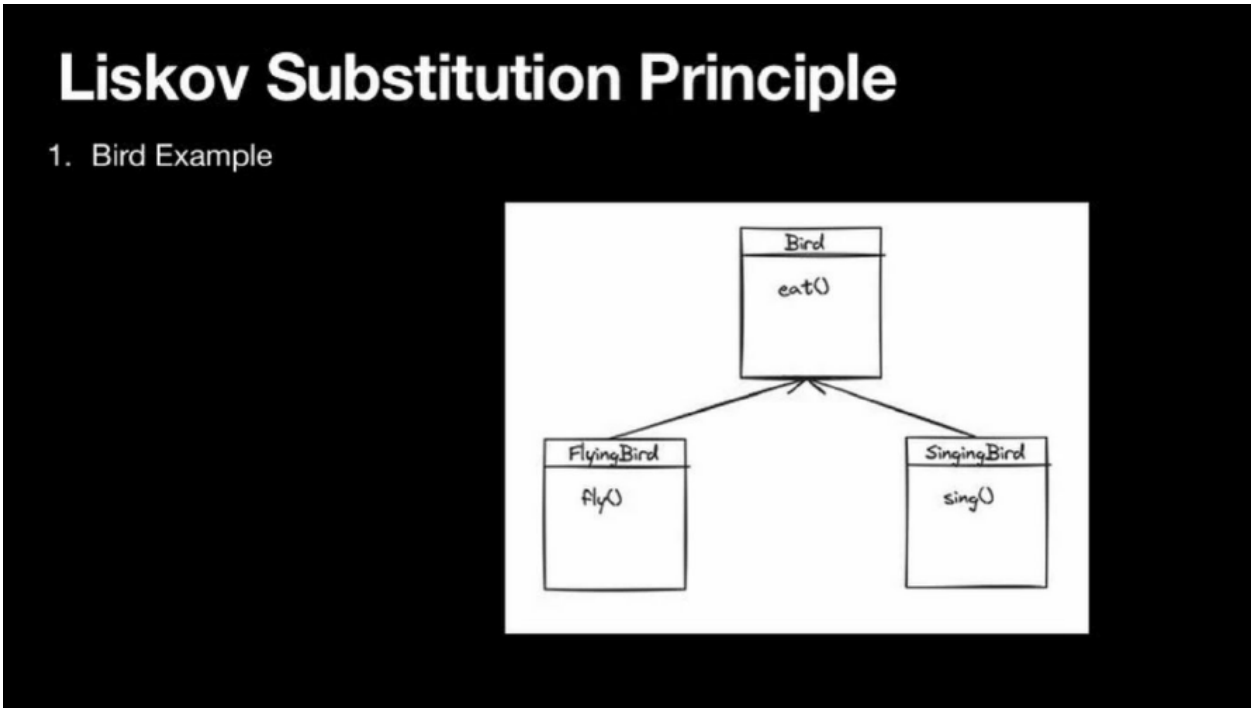



```
interface Bird {
void eat();
void sing();
void fly();
}

class Crow implements Bird {
public void eat() {
System.out.println("Crow is eating");
}

public void sing() {
    System.out.println("Crow is singing");
}

public void fly() {
    System.out.println("Crow is flying");
```

```
}

}

class Ostrich implements Bird {
public void eat() {
System.out.println("Ostrich is eating");
}

public void sing() {
    // Ostrich cannot sing
    throw new UnsupportedOperationException("Ostrich cannot sing
}

public void fly() {
    // Ostrich cannot fly
    throw new UnsupportedOperationException("Ostrich cannot fly"
}
}

public class Main {
public static void main(String[] args) {
        Bird crow = new Crow();
        crow.eat();
        crow.sing();
        crow.fly();
    Bird ostrich = new Ostrich();
    ostrich.eat();
    ostrich.sing(); // This will throw UnsupportedOperationExcep
    ostrich.fly(); // This will throw UnsupportedOperationExcept
}

}
```

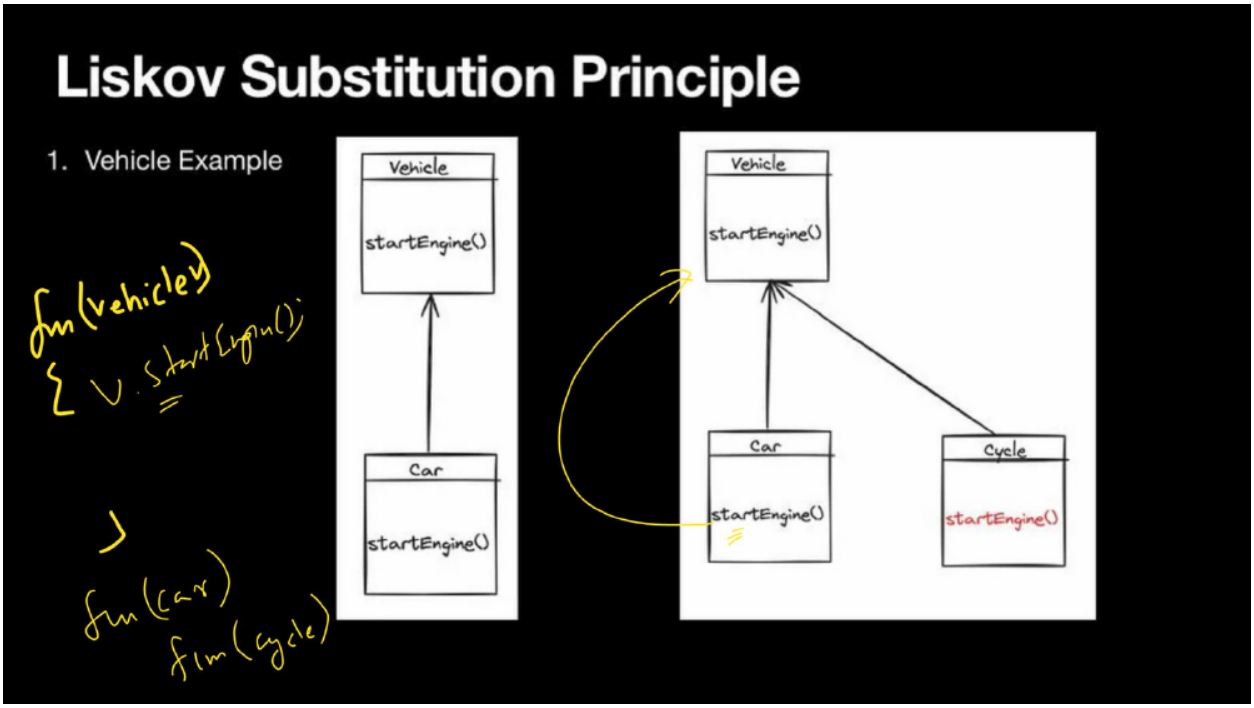
In this example, the `Crow` class implements the `Bird` interface in a way that respects all three methods: `eat`, `sing`, and `fly`. However, the `Ostrich` class violates the Liskov Substitution Principle by throwing `UnsupportedOperationException` for the `sing` and `fly` methods, indicating that an `Ostrich` cannot perform these actions. This violates the principle because clients expecting a `Bird` should be able to rely on all the methods being supported without exceptions.

Segregate Interfaces as per subclass needs




```
class Ostrich implements Bird {
  public void eat() {
     System.out.println("Ostrich is eating");
  }
}
```

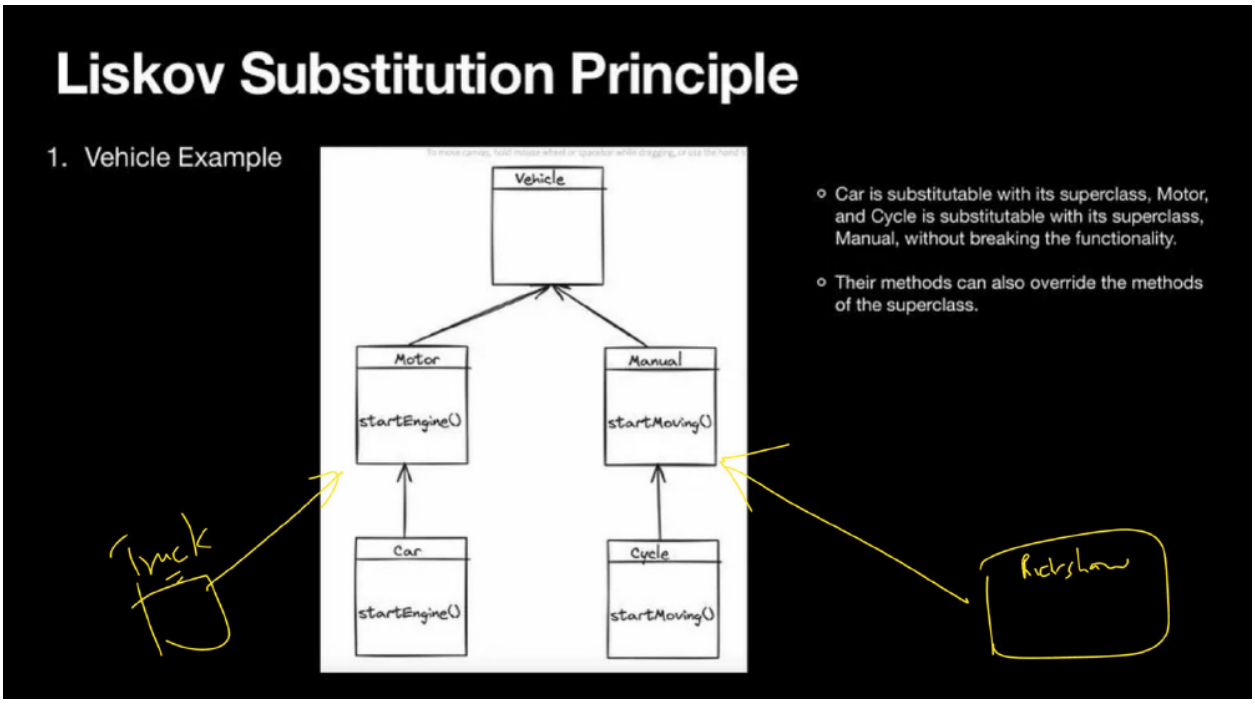
Use Case 2 : Vehicle Example




```
interface Vehicle {
   void startEngine();
}

class Car implements Vehicle {
   public void startEngine() {
     System.out.println("Car engine started");
   }
}

class Cycle implements Vehicle {
   public void startEngine() {
      throw new UnsupportedOperationException("Cycle does not ha
   }
}
```

```
public class Main {
   public static void main(String[] args) {
      Vehicle car = new Car();
       car.startEngine(); // Output: Car engine started

       Vehicle cycle = new Cycle();

       cycle.startEngine(); // This will throw UnsupportedOperat

}
```

In this example, the `Car` class implements the `Vehicle` interface with a `startEngine` method that starts the car's engine. However, the `Cycle` class violates the Liskov Substitution Principle by throwing an `UnsupportedOperationException` for the `startEngine` method, indicating that a cycle does not have an engine to start. This violates the principle because clients expecting a `Vehicle` should be able to rely on the `startEngine` method being supported for all vehicles without exceptions.




```
interface Vehicle {
    void start();
}

interface Manual {
    void startMoving();
}

interface Motor {
    void startEngine();
}

class Car implements Vehicle, Motor {
    public void start() {
        startEngine();
    }

    public void startEngine() {
```

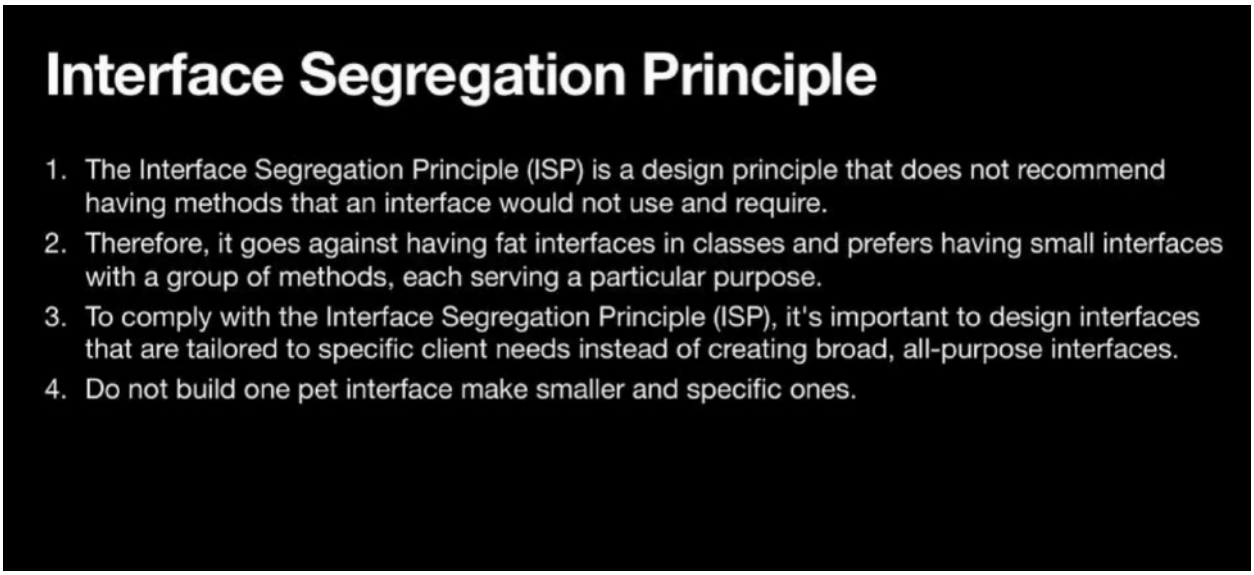
```
        System.out.println("Car engine started");
    }
}

class Cycle implements Vehicle, Manual {
    public void start() {
        startMoving();
    }

    public void startMoving() {
        System.out.println("Cycle is moving");
    }
}

public class Main {
    public static void main(String[] args) {
        Vehicle car = new Car();
        car.start(); // Output: Car engine started, Car is movi

        Vehicle cycle = new Cycle();
        cycle.start(); // Output: Cycle is moving
    }
}
```

In this revised code, the `Vehicle` interface contains a `start` method that delegates to specific methods in the `Manual` and `Motor` interfaces, allowing different types of vehicles to implement their unique start behaviours without violating the Liskov Substitution Principle.

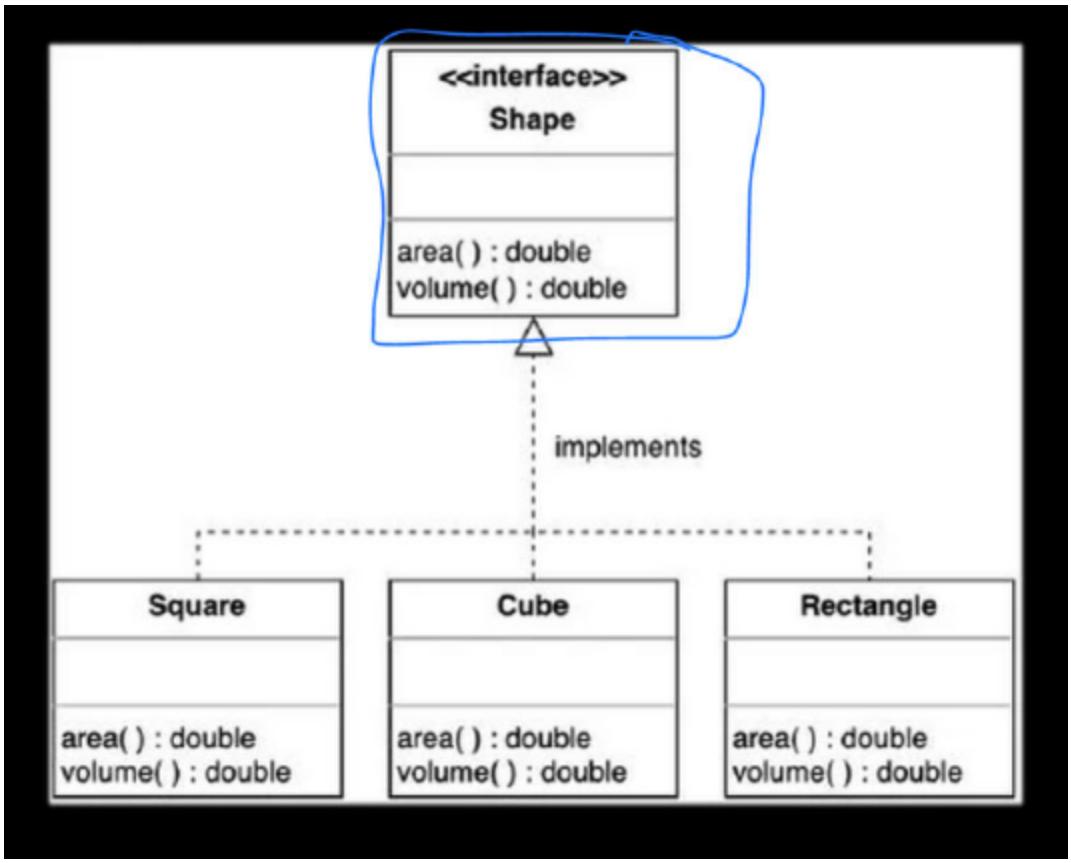
Interface Segregation Principle

The Interface Segregation Principle (ISP) is one of the five SOLID principles of object-oriented design. It states that a client should not be forced to implement an interface that it doesn't use. In other words, classes that implement interfaces should not be forced to implement methods they do not need.

The goal of the ISP is to create interfaces that are cohesive and have a single responsibility. This helps in keeping the interfaces small, focused, and easier to maintain. By adhering to the ISP, you can avoid bloated interfaces with unnecessary methods, which can lead to classes being forced to implement methods they don't actually need or use.



Use Case 1: Shape Example

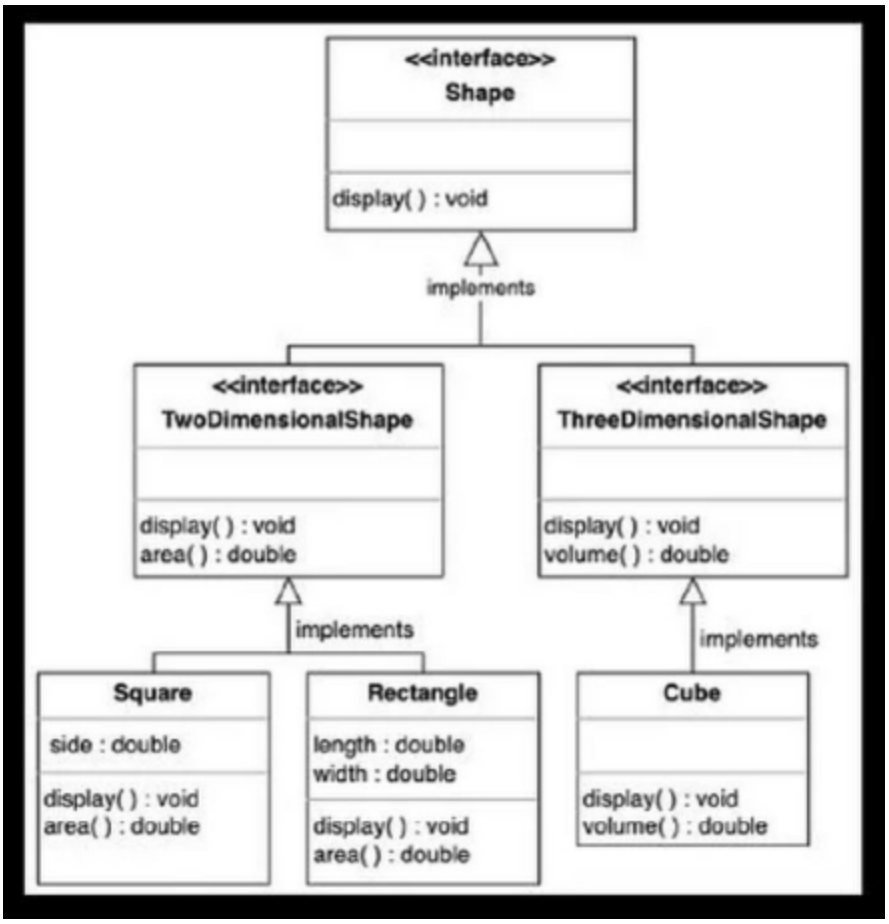



```
interface Shape {
    double area();
    double volume();
}

class Square implements Shape {
    private double side;

    public Square(double side) {
        this.side = side;
    }

    public double area() {
        return side * side;
    }
```

```
    // Violating ISP by implementing unnecessary method
    public double volume() {
        throw new UnsupportedOperationException("Square does not
    }
}

class Cube implements Shape {
    private double side;

    public Cube(double side) {
        this.side = side;
    }

    public double area() {
        return 6 * side * side;
    }

    public double volume() {
        return side * side * side;
    }
}

class Rectangle implements Shape {
    private double length;
    private double width;

    public Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }

    public double area() {
        return length * width;
    }
```

```
    // Violating ISP by implementing unnecessary method
    public double volume() {
        throw new UnsupportedOperationException("Rectangle does
    }
}

public class Main {
    public static void main(String[] args) {
        Shape square = new Square(5);
        System.out.println("Square Area: " + square.area());
        System.out.println("Square Volume: " + square.volume())

        Shape cube = new Cube(5);
        System.out.println("Cube Area: " + cube.area());
        System.out.println("Cube Volume: " + cube.volume());

        Shape rectangle = new Rectangle(5, 10);
        System.out.println("Rectangle Area: " + rectangle.area(
        System.out.println("Rectangle Volume: " + rectangle.volu
    }
}
```

Fixing the above use case




```
interface TwoDimensionalInterface {
    double area();
}

interface ThreeDimensionalInterface {
    double volume();
}

class Square implements TwoDimensionalInterface {
    private double side;

    public Square(double side) {
        this.side = side;
```

```
    }

    public double area() {
        return side * side;
    }
}

class Cube implements TwoDimensionalInterface, ThreeDimensionalI
    private double side;

    public Cube(double side) {
        this.side = side;
    }

    public double area() {
        return 6 * side * side;
    }

    public double volume() {
        return side * side * side;
    }
}

class Rectangle implements TwoDimensionalInterface {
    private double length;
    private double width;

    public Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }

    public double area() {
        return length * width;
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        TwoDimensionalInterface square = new Square(5);
        System.out.println("Square Area: " + square.area());

        ThreeDimensionalInterface cube = new Cube(5);
        System.out.println("Cube Area: " + ((Cube) cube).area()
        System.out.println("Cube Volume: " + cube.volume());

        TwoDimensionalInterface rectangle = new Rectangle(5, 10
        System.out.println("Rectangle Area: " + rectangle.area(
    }
}
```

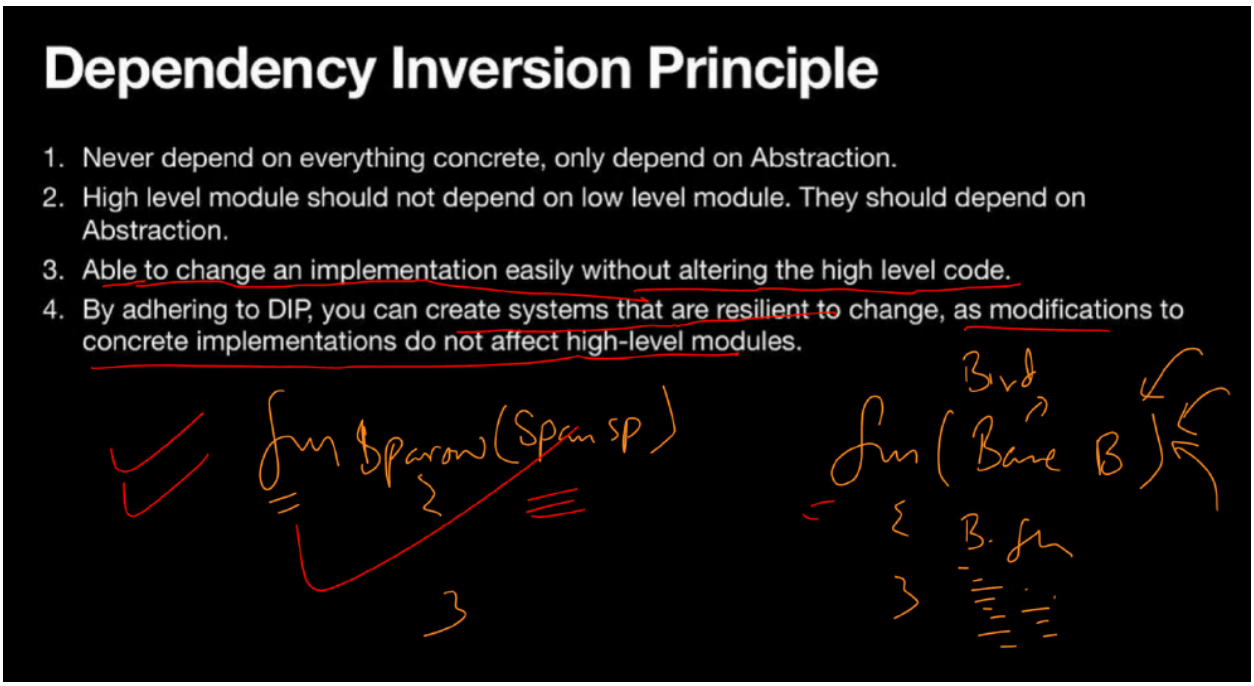
In this updated code, the `Square` class implements `TwoDimensionalInterface` for shapes with an area, the `Cube` class implements both `TwoDimensionalInterface` and `ThreeDimensionalInterface` for shapes with both an area and a volume, and the `Rectangle` class implements `TwoDimensionalInterface` for shapes with an area. Each class now implements only the interface(s) that are relevant to its behavior, adhering to the Interface Segregation Principle.

Dependency Inversion Principle

The Dependency Inversion Principle (DIP) is one of the five SOLID principles of object-oriented design. It states that high-level modules/classes should not depend on low-level modules/classes. Both should depend on abstractions. Additionally, abstractions should not depend on details. Details should depend on abstractions.

In simpler terms, the DIP suggests that classes should depend on abstractions (interfaces or abstract classes) rather than concrete implementations. This helps

in reducing the coupling between classes and allows for easier changes and maintenance.



Use Case 1: Logger

To demonstrate a violation of the Dependency Inversion Principle (DIP) in Java, we can create a scenario where an application class directly depends on a concrete class (`FileLogger`) instead of depending on an abstraction (an interface). Here's an example:

```
class FileLogger {
    public void log(String message) {
        // Implementation to log message to a file
        System.out.println("Logging to file: " + message);
    }
}

class Application {
    private FileLogger logger;
```

```
    public Application() {
        this.logger = new FileLogger();
    }

    public void doSomething() {
        // Business logic
        logger.log("Doing something");
    }
}

public class Main {
    public static void main(String[] args) {
        Application app = new Application();
        app.doSomething();
    }
}
```

In this example, the `Application` class directly depends on the `FileLogger` concrete class to perform logging. This violates the DIP because `Application` should depend on abstractions (interfaces) rather than concrete implementations.

To adhere to the DIP, we should introduce an interface (`Logger`) and make `FileLogger` implement it. Then, `Application` should depend on the `Logger` interface instead. Here's the revised example:

```
interface Logger {
    void log(String message);
}

class FileLogger implements Logger {
    public void log(String message) {
        // Implementation to log message to a file
        System.out.println("Logging to file: " + message);
    }
```

```
}

class Application {
    private Logger logger;

    public Application(Logger logger) {
        this.logger = logger;
    }

    public void doSomething() {
        // Business logic
        logger.log("Doing something");
    }
}

public class Main {
    public static void main(String[] args) {
        Logger logger = new FileLogger();
        Application app = new Application(logger);
        app.doSomething();
    }
}
```

In this revised example, `Application` now depends on the `Logger` interface, and the `FileLogger` class implements this interface. This adheres to the DIP by allowing `Application` to depend on an abstraction (`Logger`) rather than a concrete implementation (`FileLogger`).
